Arhitectura Calculatoarelor

Proiect: Implementare Procesor MIPS 32

Arhitectură Single-Cycle

Problemă: Suma numerelor puteri ale lui 2 dintr-un interval $[a, b]$

Student:
Măgerușan Alexandru

Data:
2 mai 2026

Universitatea Politehnica din Cluj-Napoca
Facultatea de Automatică și Calculatoare

Cuprins

1 Introducere

Acest proiect prezintă proiectarea hardware a unui microprocesor MIPS pe 32 de biți, Single-Cycle, implementat în VHDL. Sistemul este validat prin rezolvarea unei probleme specifice: calcularea sumei puterilor lui 2 dintr-un set de date, condiționate de apartenența la un interval dat.

2 Algoritmul de Rezolvare

2.1 Cod Sursă C++

Implementarea de referință a algoritmului în limbaj de nivel înalt.

```cpp
#include <stdio.h>

int main()
{
    int a = 2;
    int b = 90;
    int x[] = {0, 1, 2, 3, 256, 14, 16};
    int n = 7;

    int sum = 0;
    int i = -1;

    while (i < n - 1)
    {
        i++;
        int val = x[i];

        if (val < a) continue;
        if (val > b) continue;
        if (val <= 0) continue;

        int temp = val - 1;
        int test = val & temp;

        if (test != 0) continue;

        sum = sum + val;
    }

    printf("Suma:%d\n", sum);
    return 0;
}
```

Programul de referință în C++

2.2 Cod de Asamblare x86

Codul x86 generat pentru testarea logicii de procesare.

```
main:
        push    rbp
        mov     rbp, rsp
        mov     DWORD PTR [rbp-12], 2
        mov     DWORD PTR [rbp-16], 90
        mov     DWORD PTR [rbp-64], 0
        mov     DWORD PTR [rbp-60], 1
        mov     DWORD PTR [rbp-56], 2
        mov     DWORD PTR [rbp-52], 3
        mov     DWORD PTR [rbp-48], 256
        mov     DWORD PTR [rbp-44], 14
        mov     DWORD PTR [rbp-40], 16
        mov     DWORD PTR [rbp-20], 7
        mov     DWORD PTR [rbp-4], 0
        mov     DWORD PTR [rbp-8], -1
        jmp     .L2
.L7:
        add     DWORD PTR [rbp-8], 1
        mov     eax, DWORD PTR [rbp-8]
        cdqe
        mov     eax, DWORD PTR [rbp-64+rax*4]
        mov     DWORD PTR [rbp-24], eax
        mov     eax, DWORD PTR [rbp-24]
        cmp     eax, DWORD PTR [rbp-12]
        jge     .L3
        jmp     .L2
.L3:
        mov     eax, DWORD PTR [rbp-24]
        cmp     eax, DWORD PTR [rbp-16]
        jle     .L4
        jmp     .L2
.L4:
        cmp     DWORD PTR [rbp-24], 0
        jg      .L5
        jmp     .L2
.L5:
        mov     eax, DWORD PTR [rbp-24]
        sub     eax, 1
        mov     DWORD PTR [rbp-28], eax
        mov     eax, DWORD PTR [rbp-24]
        and     eax, DWORD PTR [rbp-28]
        mov     DWORD PTR [rbp-32], eax
        cmp     DWORD PTR [rbp-32], 0
        je      .L6
        jmp     .L2
.L6:
        mov     eax, DWORD PTR [rbp-24]
        add     DWORD PTR [rbp-4], eax
.L2:
        mov     eax, DWORD PTR [rbp-20]
        sub     eax, 1
        cmp     DWORD PTR [rbp-8], eax
        jl      .L7
        mov     eax, 0
        pop     rbp
        ret
```

Codul de asamblare x86

2.3 Cod Asamblare MIPS

Implementarea finală destinată execuției pe procesorul MIPS.

```
.data
    # Declaram array-ul in memorie (fiecare numar ocupa 1 word = 4 bytes)
    x: .word 0, 1, 2, 3, 256, 14, 16

.text
.globl main
main:
    addi $1, $0, 2      # a = 2
    addi $2, $0, 90     # b = 90
    addi $3, $0, 7      # n = 7
    la   $4, x          # base_addr = adresa de inceput a vectorului x
    add  $5, $0, $0     # sum = 0
    add  $6, $0, $0     # i = 0

loop_start:
    slt  $7, $6, $3     # $7 = 1 daca i < n
    beq  $7, $0, end    # Daca i >= n, am terminat, sarim la 'end'

    # --- CORECTIA DE ADRESA ---
    sll  $9, $6, 2      # $9 = i * 4
    add  $9, $4, $9     # $9 = adresa elementului curent
    lw   $8, 0($9)      # val = citim x[i] in $8

    addi $6, $6, 1      # i = i + 1

    # Verificare: val < a
    slt  $7, $8, $1     # $7 = 1 daca val < a
    bne  $7, $0, loop_start

    # Verificare: val > b
    slt  $7, $2, $8     # $7 = 1 daca b < val
    bne  $7, $0, loop_start

    # Verificare: val > 0
    slt  $7, $0, $8     # $7 = 1 daca 0 < val
    beq  $7, $0, loop_start

    # Verificare: este putere a lui 2? (val & (val - 1) == 0)
    addi $9, $8, -1     # $9 = val - 1
    and  $9, $8, $9     # $9 = val & (val - 1)
    bne  $9, $0, loop_start

    # Adunare la suma
    add  $5, $5, $8     # sum = sum + val
    j    loop_start

end:
    addi $v0, $0, 10    # exit
    syscall
```

Codul de asamblare MIPS pentru procesor

3 Setul de Instrucțiuni și Semnalele de Control

Procesorul decodifică un subset de 10 instrucțiuni MIPS 32. Tabelul următor prezintă codificările binare și semnalele generate de Unitatea de Control (`UC`).

Instrucțiune	Opcode [31-26]	RegDst	ExtOp	ALUSrc	Branch	Br_ne	Jump	MemWrite	MemtoReg	RegWrite	ALUOp[1:0]	function [5-0]	ALUCtrl[2:0]
`add`	000000	1	0	0	0	0	0	0	0	1	10	100000	000 (+)
`and`	000000	1	0	0	0	0	0	0	0	1	10	100100	010 (&)
`slt`	000000	1	0	0	0	0	0	0	0	1	10	101010	011 (¡)
`sll`	000000	1	0	0	0	0	0	0	0	1	10	000000	100 (¡¡)
`addi`	001000	0	1	1	0	0	0	0	0	1	00 (+)	X	000 (+)
`lw`	100011	0	1	1	0	0	0	0	1	1	00 (+)	X	000 (+)
`sw`	101011	0	1	1	0	0	0	1	0	0	00 (+)	X	000 (+)
`beq`	000100	0	1	0	1	0	0	0	0	0	01 (-)	X	001 (-)
`bne`	000101	0	1	0	0	1	0	0	0	0	01 (-)	X	001 (-)
`j`	000010	0	0	0	0	0	1	0	0	0	00	X	000

Tabela 1: Codificarea instrucțiunilor și semnalele de control

3.1 Clasificarea Instrucțiunilor și Datapath-ul

3.1.1 Instrucțiuni de Tip R (Register)

Utilizează trei regiștri: sursele `rs`, `rt` și destinația `rd`. ALU efectuează operația indicată de `funct`.

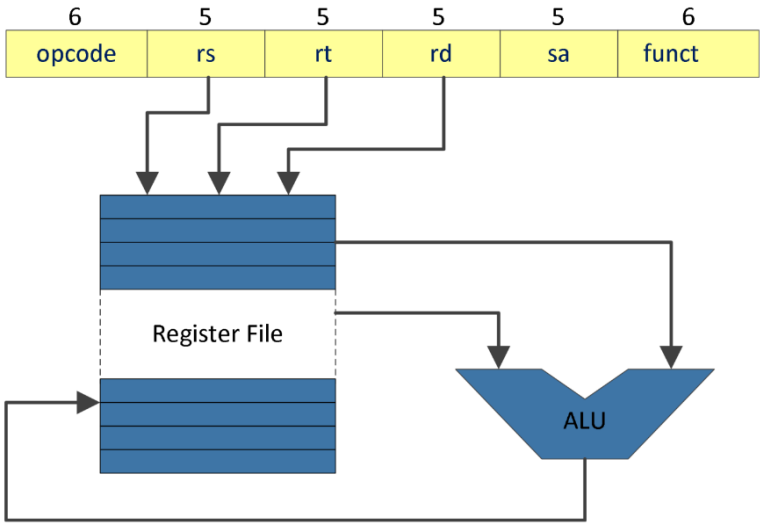

Figura 1: Calea de date pentru instrucțiunile de Tip R

3.1.2 Instrucțiuni de Tip I (Immediate)

Folosesc o constantă imediată pe 16 biți, extinsă la 32 de biți (cu semn sau cu zero) înainte de a intra în ALU.

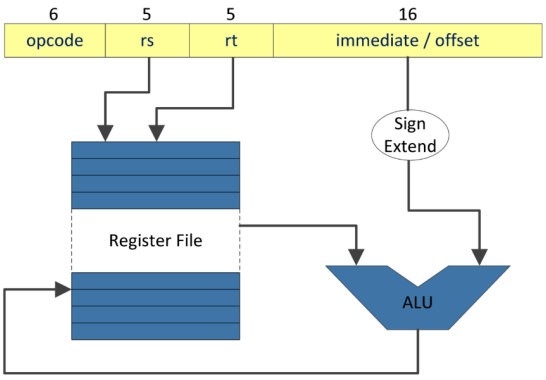

Figura 2: Sign Extend (`addi`)

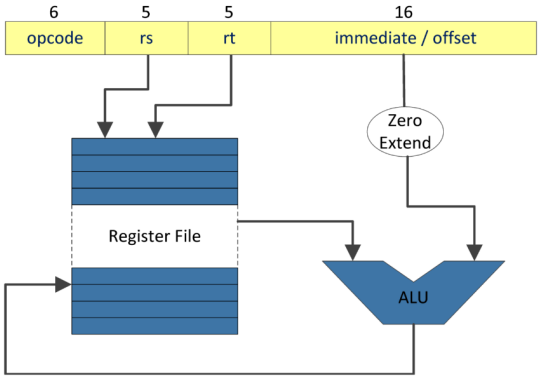

Figura 3: Zero Extend

Aritmetice și Logice cu Imediat

Acces la Memorie (`lw` și `sw`) Adresa memoriei este suma dintre registrul de bază și offset-ul extins cu semn.

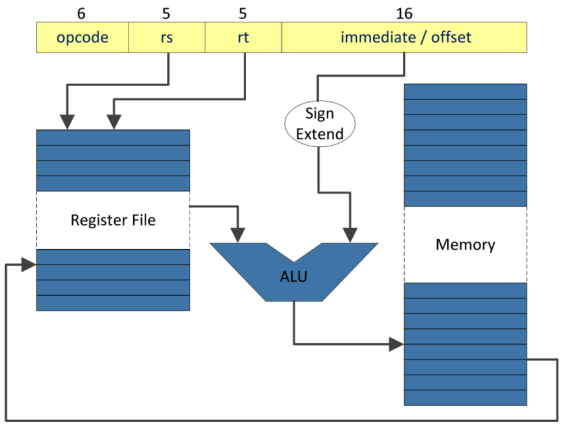

Figura 4: Load (`lw`)

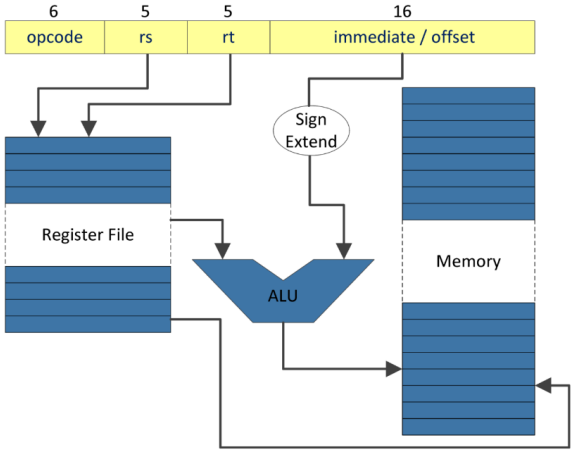

Figura 5: Store (`sw`)

Salt Condiționat (`beq`, `bne`) Decizia de salt se bazează pe verificarea egalității operanzilor în ALU.

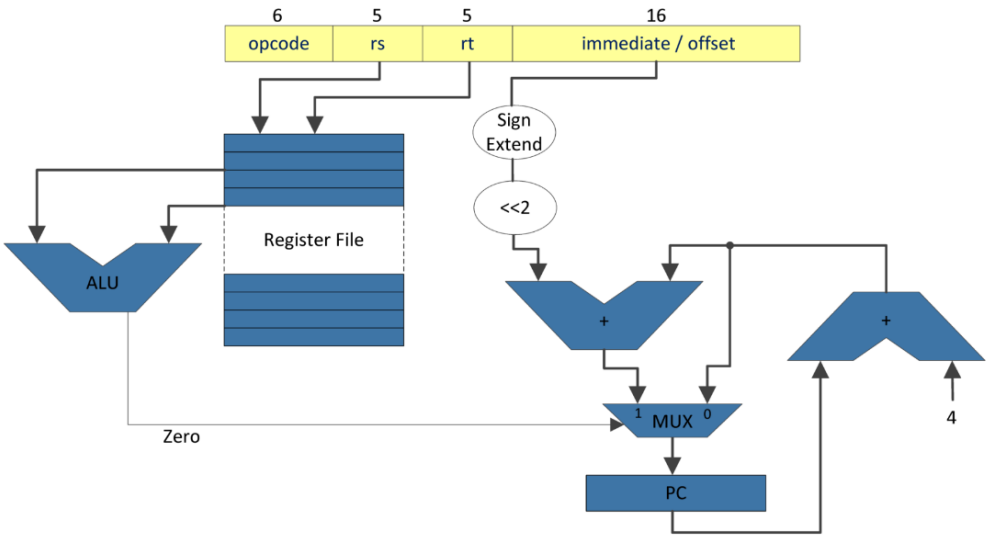

Figura 6: Instrucțiunile Branch

3.1.3 Instrucțiuni de Tip J (Jump)

Modifică direct Program Counter-ul cu adresa furnizată.

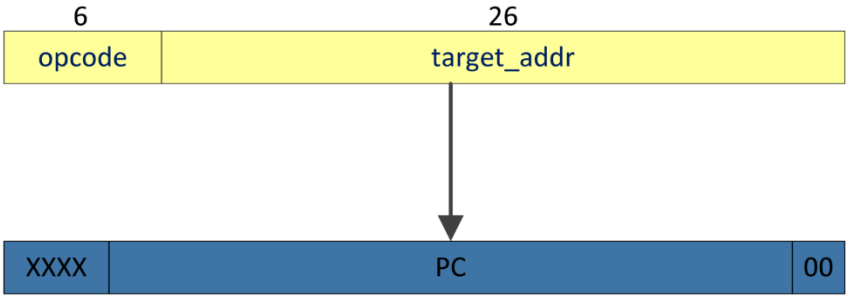

Figura 7: Instrucțiunile Jump

4 Componente Hardware și Arhitectură

4.1 Schema Bloc a Procesorului

Schema ilustrează interconectarea modulelor IFetch, IDecode, EX și MEM. A fost extinsă logic pentru suportul instrucțiunii `bne`.

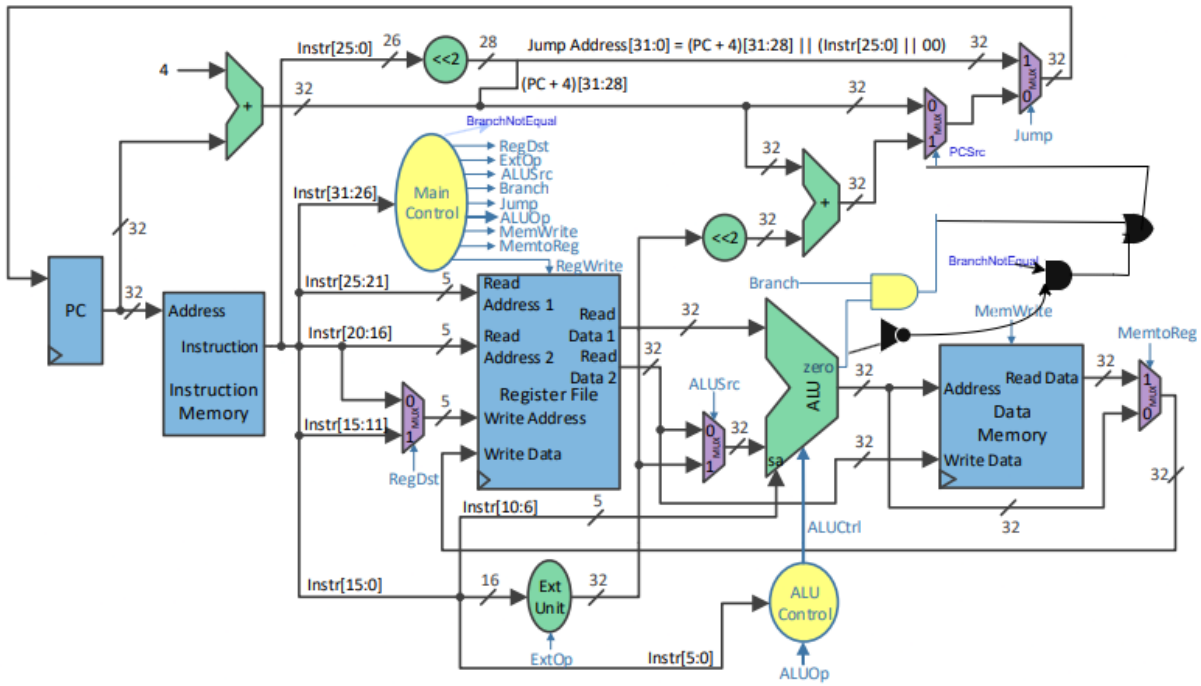

Figura 8: Arhitectura MIPS Single-Cycle

4.2 IFetch.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity IFetch is
    Port ( clk           : in  STD_LOGIC;
           rst           : in  STD_LOGIC;
           en            : in  STD_LOGIC;
           BranchAddress : in  STD_LOGIC_VECTOR (31 downto 0);
           JumpAddress   : in  STD_LOGIC_VECTOR (31 downto 0);
           Jump          : in  STD_LOGIC;
           PCSrc         : in  STD_LOGIC;
           Instruction   : out STD_LOGIC_VECTOR (31 downto 0);
           PC_plus_4     : out STD_LOGIC_VECTOR (31 downto 0));
end IFetch;

architecture Behavioral of IFetch is
    signal PC      : STD_LOGIC_VECTOR(31 downto 0) := (others => '0');
    signal Next_PC : STD_LOGIC_VECTOR(31 downto 0);
    signal PC4     : STD_LOGIC_VECTOR(31 downto 0);
    type rom_type is array (0 to 31) of std_logic_vector(31 downto 0);
    signal ROM : rom_type := (
        0  => B"000000_00000_00000_00100_00000_100000",
        1  => B"001000_00000_01010_0000000000010000",
        2  => B"100011_00100_00001_0000000000000000",
        3  => B"100011_00100_00010_0000000000000100",
        4  => B"100011_00100_00011_0000000000001000",
        5  => B"000000_00000_00000_00101_00000_100000",
        6  => B"000000_00000_00000_00110_00000_100000",
        7  => B"000000_00110_00011_00111_00000_101010",
        8  => B"000100_00111_00000_0000000000001111",
        9  => B"000000_00000_00110_01001_00010_000000",
        10 => B"000000_01010_01001_01001_00000_100000",
        11 => B"100011_01001_01000_0000000000000000",
        12 => B"001000_00110_00110_0000000000000001",
        13 => B"000000_01000_00001_00111_00000_101010",
        14 => B"000101_00111_00000_1111111111111000",
        15 => B"000000_00010_01000_00111_00000_101010",
        16 => B"000101_00111_00000_1111111111110110",
        17 => B"000000_00000_01000_00111_00000_101010",
        18 => B"000100_00111_00000_1111111111110100",
        19 => B"001000_01000_01001_1111111111111111",
        20 => B"000000_01000_01001_01001_00000_100100",
        21 => B"000101_01001_00000_1111111111110001",
        22 => B"000000_00101_01000_00101_00000_100000",
        23 => B"000010_00000000000000000000000111",
        24 => B"101011_00100_00101_0000000000001100",
        25 => B"000010_00000000000000000000011001",
        others => B"000000_00000_00000_00000_00000_000000"
    );
begin
    process(clk, rst)
    begin
        if rst = '1' then
            PC <= (others => '0');
        elsif rising_edge(clk) then
            if en = '1' then
                PC <= Next_PC;
```

```
            end if;
        end if;
    end process;

    PC4 <= PC + 4;
    PC_plus_4 <= PC4;
    Next_PC <= JumpAddress when Jump = '1' else
               BranchAddress when PCSrc = '1' else
               PC4;
    Instruction <= ROM(conv_integer(PC(6 downto 2)));
end Behavioral;
```

4.3 IDecode.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity IDecode is
    Port ( clk          : in  STD_LOGIC;
           en           : in  STD_LOGIC;
           Instr        : in  STD_LOGIC_VECTOR (25 downto 0);
           WD           : in  STD_LOGIC_VECTOR (31 downto 0);
           RegWrite     : in  STD_LOGIC;
           RegDst       : in  STD_LOGIC;
           ExtOp        : in  STD_LOGIC;
           RD1          : out STD_LOGIC_VECTOR (31 downto 0);
           RD2          : out STD_LOGIC_VECTOR (31 downto 0);
           Ext_Imm      : out STD_LOGIC_VECTOR (31 downto 0);
           func         : out STD_LOGIC_VECTOR (5 downto 0);
           sa           : out STD_LOGIC_VECTOR (4 downto 0));
end IDecode;

architecture Behavioral of IDecode is
    type reg_array is array(0 to 31) of STD_LOGIC_VECTOR(31 downto 0);
    signal reg_file : reg_array := (others => x"00000000");
    signal WriteAddress : STD_LOGIC_VECTOR(4 downto 0);
begin
    WriteAddress <= Instr(15 downto 11) when RegDst = '1' else Instr(20
       downto 16);
    process(clk)
    begin
        if rising_edge(clk) then
            if en = '1' and RegWrite = '1' then
                reg_file(conv_integer(WriteAddress)) <= WD;
            end if;
        end if;
    end process;

    RD1 <= reg_file(conv_integer(Instr(25 downto 21)));
    RD2 <= reg_file(conv_integer(Instr(20 downto 16)));
    Ext_Imm(15 downto 0) <= Instr(15 downto 0);
    Ext_Imm(31 downto 16) <= (others => Instr(15)) when ExtOp = '1' else (
       others => '0');
    func <= Instr(5 downto 0);
    sa   <= Instr(10 downto 6);
end Behavioral;
```

4.4 EX.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.NUMERIC_STD.ALL;

entity EX is
    Port ( PC_plus_4     : in  STD_LOGIC_VECTOR (31 downto 0);
           RD1           : in  STD_LOGIC_VECTOR (31 downto 0);
           RD2           : in  STD_LOGIC_VECTOR (31 downto 0);
           Ext_Imm       : in  STD_LOGIC_VECTOR (31 downto 0);
           ALUSrc        : in  STD_LOGIC;
           ALUOp         : in  STD_LOGIC_VECTOR (1 downto 0);
           func          : in  STD_LOGIC_VECTOR (5 downto 0);
           sa            : in  STD_LOGIC_VECTOR (4 downto 0);
           BranchAddress : out STD_LOGIC_VECTOR (31 downto 0);
           ALURes        : out STD_LOGIC_VECTOR (31 downto 0);
           Zero          : out STD_LOGIC);
end EX;

architecture Behavioral of EX is
    signal ALUCtrl : STD_LOGIC_VECTOR(2 downto 0);
    signal A, B    : STD_LOGIC_VECTOR(31 downto 0);
    signal ALU_Out : STD_LOGIC_VECTOR(31 downto 0);
begin
    BranchAddress <= PC_plus_4 + (Ext_Imm(29 downto 0) & "00");
    A <= RD1;
    B <= Ext_Imm when ALUSrc = '1' else RD2;

    process(ALUOp, func)
    begin
        case ALUOp is
            when "00" => ALUCtrl <= "000";
            when "01" => ALUCtrl <= "001";
            when "10" =>
                case func is
                    when "100000" => ALUCtrl <= "000";
                    when "100100" => ALUCtrl <= "010";
                    when "101010" => ALUCtrl <= "011";
                    when "000000" => ALUCtrl <= "100";
                    when others   => ALUCtrl <= "000";
                end case;
            when others => ALUCtrl <= "000";
        end case;
    end process;

    process(A, B, ALUCtrl, sa)
    begin
        case ALUCtrl is
            when "000" => ALU_Out <= A + B;
            when "001" => ALU_Out <= A - B;
            when "010" => ALU_Out <= A and B;
            when "011" =>
                if signed(A) < signed(B) then ALU_Out <= x"00000001";
                else ALU_Out <= x"00000000"; end if;
            when "100" => ALU_Out <= to_stdlogicvector(to_bitvector(B) sll
               conv_integer(sa));
            when others => ALU_Out <= (others => '0');
        end case;
```

```
58      end process;
59
60      ALURes <= ALU_Out;
61      Zero <= '1' when ALU_Out = x"00000000" else '0';
62  end Behavioral;
```

4.5 MEM.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity MEM is
    Port ( clk         : in  STD_LOGIC;
           en          : in  STD_LOGIC;
           MemWrite    : in  STD_LOGIC;
           ALUResin    : in  STD_LOGIC_VECTOR (31 downto 0);
           RD2         : in  STD_LOGIC_VECTOR (31 downto 0);
           MemData     : out STD_LOGIC_VECTOR (31 downto 0);
           ALUResout   : out STD_LOGIC_VECTOR (31 downto 0));
end MEM;

architecture Behavioral of MEM is
    type ram_type is array (0 to 63) of STD_LOGIC_VECTOR(31 downto 0);
    signal RAM : ram_type := (
        0 => x"00000002", -- RAM[0]: a = 2
        1 => x"0000005A", -- RAM[4]: b = 90
        2 => x"00000007", -- RAM[8]: n = 7
        3 => x"00000000", -- RAM[12]: sum = 0
        4 => x"00000000", -- RAM[16]: x[0] = 0
        5 => x"00000001", -- RAM[20]: x[1] = 1
        6 => x"00000002", -- RAM[24]: x[2] = 2
        7 => x"00000003", -- RAM[28]: x[3] = 3
        8 => x"00000100", -- RAM[32]: x[4] = 256
        9 => x"0000000E", -- RAM[36]: x[5] = 14
        10 => x"00000010", -- RAM[40]: x[6] = 16
        others => x"00000000"
    );

begin
    process(clk)
    begin
        if rising_edge(clk) then
            if en = '1' and MemWrite = '1' then
                RAM(conv_integer(ALUResin(7 downto 2))) <= RD2;
            end if;
        end if;
    end process;
    MemData <= RAM(conv_integer(ALUResin(7 downto 2)));
    ALUResout <= ALUResin;
end Behavioral;
```

4.6 UC.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity UC is
    Port ( Instr_opcode : in  STD_LOGIC_VECTOR (5 downto 0);
           RegDst       : out STD_LOGIC;
           ExtOp        : out STD_LOGIC;
           ALUSrc       : out STD_LOGIC;
           Branch       : out STD_LOGIC;
           BranchNotEq  : out STD_LOGIC;
           Jump         : out STD_LOGIC;
           ALUOp        : out STD_LOGIC_VECTOR (1 downto 0);
           MemWrite     : out STD_LOGIC;
           MemtoReg     : out STD_LOGIC;
           RegWrite     : out STD_LOGIC);
end UC;

architecture Behavioral of UC is
begin
    process(Instr_opcode)
    begin
        RegDst <= '0'; ExtOp <= '0'; ALUSrc <= '0'; Branch <= '0';
        BranchNotEq <= '0'; Jump <= '0'; MemWrite <= '0'; MemtoReg <= '0';
        RegWrite <= '0'; ALUOp <= "00";

        case Instr_opcode is
            when "000000" => RegDst <= '1'; RegWrite <= '1'; ALUOp <= "10";
            when "001000" => ExtOp <= '1'; ALUSrc <= '1'; RegWrite <= '1';
               ALUOp <= "00";
            when "100011" => ExtOp <= '1'; ALUSrc <= '1'; MemtoReg <= '1';
               RegWrite <= '1'; ALUOp <= "00";
            when "101011" => ExtOp <= '1'; ALUSrc <= '1'; MemWrite <= '1';
               ALUOp <= "00";
            when "000100" => ExtOp <= '1'; Branch <= '1'; ALUOp <= "01";
            when "000101" => ExtOp <= '1'; BranchNotEq <= '1'; ALUOp <= "01"
               ;
            when "000010" => Jump <= '1';
            when others => null;
        end case;
    end process;
end Behavioral;
```

4.7 test_env.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity test_env is
    Port ( clk : in  STD_LOGIC;
           btn : in  STD_LOGIC_VECTOR (4 downto 0);
           sw  : in  STD_LOGIC_VECTOR (15 downto 0);
           led : out STD_LOGIC_VECTOR (15 downto 0);
           an  : out STD_LOGIC_VECTOR (7 downto 0);
           cat : out STD_LOGIC_VECTOR (6 downto 0));
end test_env;

architecture Behavioral of test_env is

    component IFetch is
        Port ( clk, rst, en : in STD_LOGIC;
               BranchAddress, JumpAddress : in STD_LOGIC_VECTOR (31 downto
                  0);
               Jump, PCSrc : in STD_LOGIC;
               Instruction, PC_plus_4 : out STD_LOGIC_VECTOR (31 downto 0));
    end component;

    component UC is
        Port ( Instr_opcode : in STD_LOGIC_VECTOR (5 downto 0);
               RegDst, ExtOp, ALUSrc, Branch, BranchNotEq, Jump : out
                  STD_LOGIC;
               ALUOp : out STD_LOGIC_VECTOR (1 downto 0);
               MemWrite, MemtoReg, RegWrite : out STD_LOGIC);
    end component;

    component IDecode is
        Port ( clk, en : in STD_LOGIC;
               Instr : in STD_LOGIC_VECTOR (25 downto 0);
               WD : in STD_LOGIC_VECTOR (31 downto 0);
               RegWrite, RegDst, ExtOp : in STD_LOGIC;
               RD1, RD2, Ext_Imm : out STD_LOGIC_VECTOR (31 downto 0);
               func : out STD_LOGIC_VECTOR (5 downto 0);
               sa : out STD_LOGIC_VECTOR (4 downto 0));
    end component;

    component EX is
        Port ( PC_plus_4, RD1, RD2, Ext_Imm : in STD_LOGIC_VECTOR (31 downto
            0);
               ALUSrc : in STD_LOGIC;
               ALUOp : in STD_LOGIC_VECTOR (1 downto 0);
               func : in STD_LOGIC_VECTOR (5 downto 0);
               sa : in STD_LOGIC_VECTOR (4 downto 0);
               BranchAddress, ALURes : out STD_LOGIC_VECTOR (31 downto 0);
               Zero : out STD_LOGIC);
    end component;

    component MEM is
        Port ( clk, en, MemWrite : in STD_LOGIC;
               ALUResin, RD2 : in STD_LOGIC_VECTOR (31 downto 0);
               MemData, ALUResout : out STD_LOGIC_VECTOR (31 downto 0));
    end component;

```

```
56      component MPG is
57          Port ( enable : out STD_LOGIC;
58                 btn, clk : in STD_LOGIC);
59      end component;
60
61      component SSD is
62          Port ( clk : in STD_LOGIC;
63                 digits : in STD_LOGIC_VECTOR(31 downto 0);
64                 an : out STD_LOGIC_VECTOR(7 downto 0);
65                 cat : out STD_LOGIC_VECTOR(6 downto 0));
66      end component;
67
68      signal en, rst : STD_LOGIC;
69      signal Instruction, PC_plus_4 : STD_LOGIC_VECTOR(31 downto 0);
70      signal RegDst, ExtOp, ALUSrc, Branch, BranchNotEq, Jump, MemWrite,
           MemtoReg, RegWrite : STD_LOGIC;
71      signal ALUOp : STD_LOGIC_VECTOR(1 downto 0);
72      signal RD1, RD2, Ext_Imm : STD_LOGIC_VECTOR(31 downto 0);
73      signal func : STD_LOGIC_VECTOR(5 downto 0);
74      signal sa : STD_LOGIC_VECTOR(4 downto 0);
75      signal BranchAddress, ALURes, MemData, ALUResout, WD, JumpAddress :
           STD_LOGIC_VECTOR(31 downto 0);
76      signal Zero, PCSrc : STD_LOGIC;
77      signal digits_out : STD_LOGIC_VECTOR(31 downto 0);
78
79  begin
80
81      empege: MPG port map (enable => en, btn => btn(0), clk => clk);
82      rst <= btn(1);
83      display: SSD port map (clk => clk, digits => digits_out, an => an, cat
           => cat);
84
85      inst_IFetch: IFetch port map (clk, rst, en, BranchAddress, JumpAddress,
           Jump, PCSrc, Instruction, PC_plus_4);
86      inst_UC: UC port map (Instruction(31 downto 26), RegDst, ExtOp, ALUSrc,
           Branch, BranchNotEq, Jump, ALUOp, MemWrite, MemtoReg, RegWrite);
87      inst_IDecode: IDecode port map (clk, en, Instruction(25 downto 0), WD,
           RegWrite, RegDst, ExtOp, RD1, RD2, Ext_Imm, func, sa);
88      inst_EX: EX port map (PC_plus_4, RD1, RD2, Ext_Imm, ALUSrc, ALUOp, func,
            sa, BranchAddress, ALURes, Zero);
89      inst_MEM: MEM port map (clk, en, MemWrite, ALURes, RD2, MemData,
           ALUResout);
90
91      WD <= MemData when MemtoReg = '1' else ALUResout;
92      PCSrc <= (Branch and Zero) or (BranchNotEq and not Zero);
93      JumpAddress <= PC_plus_4(31 downto 28) & Instruction(25 downto 0) & "00"
           ;
94
95      led(0) <= RegDst; led(1) <= ExtOp; led(2) <= ALUSrc; led(3) <= Branch;
96      led(4) <= BranchNotEq; led(5) <= Jump; led(6) <= MemWrite; led(7) <=
           MemtoReg;
97      led(8) <= RegWrite; led(10 downto 9) <= ALUOp; led(15 downto 11) <= (
           others => '0');
98
99      process(sw(7 downto 5), Instruction, PC_plus_4, RD1, RD2, Ext_Imm,
           ALURes, MemData, WD)
100     begin
101         case sw(7 downto 5) is
102             when "000" => digits_out <= Instruction;
103             when "001" => digits_out <= PC_plus_4;
```

```
            when "010" => digits_out <= RD1;
            when "011" => digits_out <= RD2;
            when "100" => digits_out <= Ext_Imm;
            when "101" => digits_out <= ALURes;
            when "110" => digits_out <= MemData;
            when "111" => digits_out <= WD;
            when others => digits_out <= (others => '0');
        end case;
    end process;

end Behavioral;
```

5 Cod Mașină și Execuție

PC	Cod Mașină (Binar)	Hexa	MIPS ASM
0	001000_00000_00001_0000000000000010	20010002	`addi $1, $0, 2`
1	001000_00000_00010_0000000001011010	2002005A	`addi $2, $0, 90`
2	001000_00000_00011_0000000000000111	20030007	`addi $3, $0, 7`
3	000000_00000_00000_00100_00000_100000	00002020	`add $4, $0, $0` *(la $4, x)*
4	000000_00000_00000_00101_00000_100000	00002820	`add $5, $0, $0`
5	000000_00000_00000_00110_00000_100000	00003020	`add $6, $0, $0`
— loop_start: (PC = 6) —			
6	000000_00110_00011_00111_00000_101010	00C3382A	`slt $7, $6, $3`
7	000100_00111_00000_0000000000001111	10E0000F	`beq $7, $0, end` *(+15)*
8	000000_00000_00110_01001_00010_000000	00064880	`sll $9, $6, 2`
9	000000_00100_01001_01001_00000_100000	00894820	`add $9, $4, $9`
10	100011_01001_01000_0000000000000000	8D280000	`lw $8, 0($9)`
11	001000_00110_00110_0000000000000001	20C60001	`addi $6, $6, 1`
12	000000_01000_00001_00111_00000_101010	0101382A	`slt $7, $8, $1`
13	000101_00111_00000_1111111111111000	14E0FFF8	`bne $7, $0, loop_start` *(-8)*
14	000000_00010_01000_00111_00000_101010	0048382A	`slt $7, $2, $8`
15	000101_00111_00000_1111111111110110	14E0FFF6	`bne $7, $0, loop_start` *(-10)*
16	000000_00000_01000_00111_00000_101010	0008382A	`slt $7, $0, $8`
17	000100_00111_00000_1111111111110100	10E0FFF4	`beq $7, $0, loop_start` *(-12)*
18	001000_01000_01001_1111111111111111	2109FFFF	`addi $9, $8, -1`
19	000000_01000_01001_01001_00000_100100	01094824	`and $9, $8, $9`
20	000101_01001_00000_1111111111110001	1520FFF1	`bne $9, $0, loop_start` *(-15)*
21	000000_00101_01000_00101_00000_100000	00A82820	`add $5, $5, $8`
22	000010_00000000000000000000000110	08000006	`j loop_start` *(Jmp target 6)*
— end: (PC = 23) —			
23	001000_00000_00010_0000000000001010	2002000A	`addi $v0, $0, 10` *($v0 is $2)*
24	000000_00000_00000_00000_00000_001100	0000000C	`syscall` *(cod generic syscall)*

Tabela 2: Mapping-ul detaliat între codul sursă și memoria ROM

6 Trasarea Execuției (Execution Trace)

În prima iterație, indexul $i = 0$, iar valoarea citită din memorie este $x[0] = 0$.

Pas	SW(7:5)	"000"	"001"	"010"	"011"	"100"	"101"	"110"	"111"	Numai pt. salt	
	Instr (asamblare)	Instr (hexa)	PC+4	RD1	RD2	Ext_Imm	ALURes	MemData	WD	BranchAddr	JumpAddr
0	addi $1, $0, 2	20010002	00000004	00000000	00000000	00000002	00000002	XXXXXXXX	00000002	XXXXXXXX	XXXXXXXX
1	addi $2, $0, 90	2002005A	00000008	00000000	00000000	0000005A	0000005A	XXXXXXXX	0000005A	XXXXXXXX	XXXXXXXX
2	addi $3, $0, 7	20030007	0000000C	00000000	00000000	00000007	00000007	XXXXXXXX	00000007	XXXXXXXX	XXXXXXXX
3	add $4, $0, $0	00002020	00000010	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
4	add $5, $0, $0	00002820	00000014	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
5	add $6, $0, $0	00003020	00000018	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
6	slt $7, $6, $3	00C3382A	0000001C	00000000	00000007	XXXXXXXX	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
7	beq $7, $0, end	10E0000F	00000020	00000001	00000000	0000000F	00000001	XXXXXXXX	XXXXXXXX	0000005C	XXXXXXXX
8	sll $9, $6, 2	00064880	00000024	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
9	add $9, $4, $9	00894820	00000028	00000000	00000000	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
10	lw $8, 0($9)	8D280000	0000002C	00000000	00000000	00000000	00000000	00000002	00000002	XXXXXXXX	XXXXXXXX
11	addi $6, $6, 1	20C60001	00000030	00000000	XXXXXXXX	00000001	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
12	slt $7, $8, $1	0101382A	00000034	00000002	00000002	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
13	bne $7, $0, loop	14E0FFF8	00000038	00000000	00000000	FFFFFFF8	00000000	XXXXXXXX	XXXXXXXX	00000018	XXXXXXXX
14	slt $7, $2, $8	0048382A	0000003C	0000005A	00000002	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
15	bne $7, $0, loop	14E0FFF6	00000040	00000000	00000000	FFFFFFF6	00000000	XXXXXXXX	XXXXXXXX	00000018	XXXXXXXX
16	slt $7, $0, $8	0008382A	00000044	00000000	00000002	XXXXXXXX	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
17	beq $7, $0, loop	10E0FFF4	00000048	00000001	00000000	FFFFFFF4	00000001	XXXXXXXX	XXXXXXXX	00000018	XXXXXXXX
18	addi $9, $8, -1	2109FFFF	0000004C	00000002	XXXXXXXX	FFFFFFFF	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX
19	and $9, $8, $9	01094824	00000050	00000002	00000001	XXXXXXXX	00000000	XXXXXXXX	00000000	XXXXXXXX	XXXXXXXX
20	bne $9, $0, loop	1520FFF1	00000054	00000000	00000000	FFFFFFF1	00000000	XXXXXXXX	XXXXXXXX	00000018	XXXXXXXX
21	add $5, $5, $8	00A82820	00000058	00000000	00000002	XXXXXXXX	00000002	XXXXXXXX	00000002	XXXXXXXX	XXXXXXXX
22	j loop_start	08000006	0000005C	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	XXXXXXXX	00000018
23	slt $7, $6, $3	00C3382A	0000001C	00000001	00000007	XXXXXXXX	00000001	XXXXXXXX	00000001	XXXXXXXX	XXXXXXXX

Tabela 3: Trasarea execuției (Prima iterație completă)

Notă privind trasarea: Tabelul reflectă starea datapath-ului la execuția fiecărei instrucțiuni. Valorile marcate cu "XXXXXXXX" reprezintă semnale tip "Don't Care" pentru instrucțiunea respectivă.

7 Sinteză FPGA

Sinteza proiectului în mediul Vivado s-a finalizat cu succes, generând fișierul `test_env.bit`. Configurația a fost programată pe FPGA.